

Relatório – Atividade 6: Vetorização

Integrantes: Luiz Felliipe Catuzzi 11871198

Juan Henriques Passos 15464826

Fernando Valentim Torres 15452340

1. Objetivo

Testar as instruções *intrinsics* para vetorização do código, conforme visto na aula 6. Para isso, foi utilizado um código que faz uso intensivo de matrizes, comparando uma versão escalar padrão com uma versão vetorizada manualmente.

2. Metodologia

O algoritmo escolhido foi a Multiplicação de Matrizes (GEMM), que implementa a operação $C = A * B + C$.

Para a análise, foram criadas duas versões do executável a partir de dois códigos-fonte (`matrix_scalar.c` e `matrix_avx.c`):

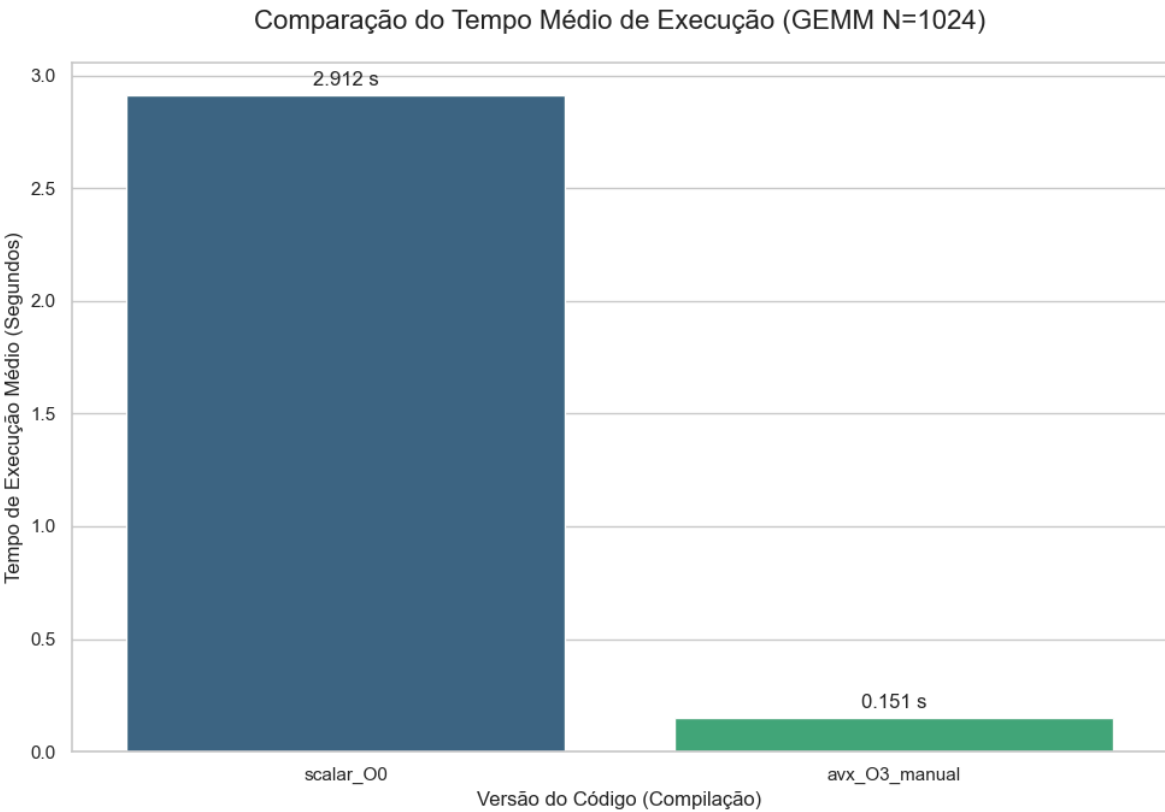
1. Escalar Pura (`scalar_00`): Compilado com `-O0` para desabilitar todas as otimizações do compilador. Esta é a nossa linha de base (*baseline*) para medir o desempenho de um *loop* `ijk` simples.
2. Vetorizada Manual (`avx_03_manual`): Compilado com `-O3 -mavx -mfma`. Esta versão utiliza instruções intrínsecas AVX e FMA (como `_mm256_fmadd_ps`) para processar 8 *floats* (256 bits) por vez no *loop* mais interno.

Para assegurar a consistência estatística dos resultados, os experimentos foram padronizados em 10 execuções para cada versão do código (escalar e vetorizada), registrando-se a média aritmética das métricas obtidas. Para a avaliação de desempenho temporal, utilizou-se uma matriz de dimensão $N=1024$ como carga de trabalho. De maneira análoga, para a análise de eficiência de memória, empregou-se a ferramenta de perfilamento Linux `perf` monitorando os eventos de hardware cache-references, cache-misses e L1-dcache-load-misses. Essa abordagem permitiu comparar o impacto das instruções AVX (256 bits) tanto no tempo de execução quanto na eficiência da localidade espacial e temporal, correlacionando o *speedup* obtido com a variação na taxa de falhas de cache (*miss rate*).

3. Resultados Obtidos

3.1 Tempo de Execução (N=1024)

Versão	Tempo Médio (10 execuções)	Speedup (vs. Escalar O0)
scalar_O0	2.912118 s	1.0x
avx_O3_manual	0.150660 s	19.329x



3.2 Análise de Cache (N=256)

Métrica	Escalar (Baseline)	Vetorizada (AVX)	Diferença / Impacto

L1-dcache Loads (Total de Acessos L1)	2.099.766.880	36.847.241	-98,2% (Redução Massiva)
L1-dcache Misses (Falhas na L1)	2.894.543	6.679.971	+130% (Aumentou*)
Taxa de Falha L1 (Miss Rate)	~0,13%	~18,1%	N/A
LLC Misses (Falhas na Cache de Último Nível)	602.453	455.329	-24,4% (Melhorou)

4. Análise dos Resultados

A análise dos dados revela o impacto massivo da vetorização em operações de matriz.

Análise do Tempo de Execução (N=1024):

1. `scalar_00` (Linha de Base): O tempo de 2.912118s representa o pior cenário. O código processa um único *float* (4 bytes) por vez, sendo limitado pela latência da memória e pela sobrecarga dos *loops*.
2. `avx_03_manual` (Vetorização Manual): Nossa implementação com *intrinsics* manuais (AVX/FMA) alcançou um *speedup* massivo de 19.329x, reduzindo o tempo de execução para 0.150660s.

Análise de Acesso à Cache (N=256):

- `scalar_00` (Ineficiência de Acesso): A versão escalar exigiu impressionantes 2.09 bilhões de acessos à cache L1 (L1-dcache-loads). Apesar de ter uma taxa de falhas baixa (0,13%) — indicando que o *prefetcher* previu bem os acessos lineares — o gargalo foi o volume excessivo de instruções. A CPU gastou a maior parte dos ciclos apenas despachando instruções de leitura de memória individualmente.
- `avx_03_manual` (Eficiência de Largura de Banda): A versão vetorizada reduziu o número de acessos à memória para 36.8 milhões, uma redução de 98.2% no tráfego de instruções de carga. Ao carregar 8 *floats* (32 bytes) em uma única instrução, o código vetorizado otimizou o uso da linha de cache (64 bytes).
 - *Nota sobre Cache Misses*: Observou-se um aumento absoluto no número de *L1 misses* na versão vetorizada (de ~2.8M para ~6.6M). Isso é um comportamento esperado em algoritmos de alta vazão: como a CPU processa os dados 19x mais rápido, ela pressiona a hierarquia de memória

com muito mais intensidade em um curto espaço de tempo, saturando a capacidade da L1 mais rapidamente do que a versão lenta escalar.

5. Conclusões

A partir dos testes, concluímos uma flagrante melhora de desempenho de tempo de execução (um *speedup* de mais de 19x) à partir da vetorização.

A vetorização manual com instruções *intrinsics* (AVX/FMA) provou ser uma forma de alcançar alto desempenho, transformando um problema limitado pela latência (Escalar) em um problema limitado pela largura de banda (Vetorizado). Isso demonstra que o conhecimento profundo da arquitetura de hardware (SIMD) e o uso de instruções específicas (como `_mm256_fmadd_ps`) são essenciais para extrair o desempenho máximo de códigos de computação intensiva, como a multiplicação de matrizes.

Conclui-se que a vetorização não apenas acelerou o cálculo matemático (FLOPS), mas principalmente otimizou a largura de banda da memória. Ao reduzir a quantidade de acessos à cache (Loads) em mais de 50 vezes, o código vetorizado removeu o gargalo de emissão de instruções da CPU, permitindo atingir o *speedup* de 19x observado.